

Mineração de Padrões: Apriori, FP Growth e regras úteis

O Que é Análise de Padrões Frequentes?

Padrão Frequente

Um conjunto de itens, subsequências ou subestruturas que ocorre frequentemente em um conjunto de dados

Mineração de Padrões

Foco na descoberta de padrões que aparecem com alta frequência nos dados

Regras de Associação

Etapas subsequentes que derivam regras de implicação a partir dos padrões frequentes descobertos

Questões de Negócio

- Quais produtos são frequentemente comprados juntos?
- Cerveja e fraldas aparecem juntos?
- Quais são as compras subsequentes após adquirir um PC?

Aplicações Práticas

- Análise de cesta de compras
- Marketing cruzado e design de catálogos
- Análise de campanhas de vendas
- Análise de logs Web (clickstream)
- Análise de sequências de DNA

Conceitos Básicos: Dados transacionais

Vamos explorar um exemplo prático de banco de dados transacional para entender como padrões frequentes emergem de transações reais de compras.

Tid	Itens Comprados
10	Cerveja, Nozes, Fralda
20	Cerveja, Café, Fralda
30	Cerveja, Fralda, Ovos
40	Nozes, Ovos, Leite
50	Nozes, Café, Fralda, Ovos, Leite

Neste exemplo de transações de supermercado, podemos observar que certos itens aparecem juntos com frequência. Por exemplo, Cerveja e Fralda aparecem em três transações (10, 20, 30), representando um padrão frequente interessante.

Usando a notação estatística (*), podemos calcular o suporte de cada padrão: a frequência com que ele aparece no conjunto de dados.

 **Notação**

Tid = ID da Transação

Suporte = Número de transações contendo o padrão

Conceitos Básicos: Padrões Frequentes

Padrões frequentes são conjuntos de itens que aparecem juntos com frequência em uma base de dados transacional. A mineração de padrões frequentes é fundamental para descobrir associações interessantes entre itens em grandes volumes de dados.

Exemplo de Base de Dados Transacional

Tid	Itens Comprados
10	Cerveja, Nozes, Fralda
20	Cerveja, Café, Fralda
30	Cerveja, Fralda, Ovos
40	Nozes, Ovos, Leite
50	Nozes, Café, Fralda, Ovos, Leite

Padrão Frequente

Um conjunto de itens X que aparece em pelo menos σ transações, onde σ é o suporte mínimo definido (*)

Suporte

Percentual de transações que contêm um determinado conjunto de itens:
$$\text{sup}(X) = |DX|/|D|$$

Conceitos Básicos: Regras de Associação

Regras de associação expressam relações de implicação entre conjuntos de itens na forma $X \rightarrow Y$, onde X e Y são conjuntos disjuntos de itens. Essas regras revelam padrões do tipo "se um cliente compra X, então provavelmente comprará Y".

Base de Dados

Tid	Itens
10	Cerveja, Nozes, Fralda
20	Cerveja, Café, Fralda
30	Cerveja, Fralda, Ovos
40	Nozes, Ovos, Leite
50	Nozes, Café, Fralda, Ovos, Leite

Exemplo de Regra

{Cerveja} → {Fralda}

Esta regra indica que clientes que compram cerveja tendem a comprar fraldas também.

Suporte: 60% (3 de 5 transações)

Confiança: 100% (todas as transações com cerveja têm fralda)

Valores de lift maiores que 1 indicam correlação positiva, enquanto valores menores que 1 sugerem correlação negativa entre os itens.

Confiança

Probabilidade condicional de Y dado X: $\text{conf}(X \rightarrow Y) = \frac{\text{sup}(X \cup Y)}{\text{sup}(X)}$

Suporte

Frequência da regra completa na base de dados: $\text{sup}(X \rightarrow Y) = \text{sup}(X \cup Y)$

Lift

Mede a força da associação: $\text{lift}(X \rightarrow Y) = \frac{\text{conf}(X \rightarrow Y)}{\text{sup}(Y)}$

Desafio: Existem Padrões Frequentes Demais!

Um padrão longo contém um número combinatório de sub-padrões, criando uma explosão computacional que torna a mineração inviável sem técnicas avançadas de compressão.

Quantos conjuntos de itens frequentes o seguinte TDBI contém com suporte mínimo = 1?

TDBI: T1: {a1, ..., a50}; T2: {a1, ..., a100}

1-itemsets

{a1}: 2, {a2}: 2, ..., {a50}: 2, {a51}: 1, ..., {a100}: 1

2-itemsets

{a1, a2}: 2, ..., {a1, a50}: 2, {a1, a51}: 1, ..., {a99, a100}: 1

99-itemsets

{a1, a2, ..., a99}: 1, ..., {a2, a3, ..., a100}: 1

100-itemset

{a1, a2, ..., a100}: 1

Número Total de Conjuntos Frequentes

$$(2^{50} - 1) \times 2 + 2^{50} - 1 = 2^{51} + 2^{50} - 3 \approx 3.38 \times 10^{15}$$

Um conjunto imenso demais para qualquer pessoa computar ou armazenar!

Expressando Padrões de Forma Comprimida: Padrões Fechados

Como Lidar com Este Desafio?

Solução 1: Padrões Fechados

Um padrão (conjunto de itens) X é fechado se X é frequente e não existe nenhum super-padrão $Y \supseteq X$ com o mesmo suporte que X .

❏ **Exemplo:** Seja TDBI: T1: {a1, ..., a50}; T2: {a1, ..., a100}

Com suporte mínimo = 1, quantos padrões fechados TDBI contém?

Dois Padrões

P1: "{a1, ..., a50}: 2"

P2: "{a1, ..., a100}: 1"

Compressão Sem Perdas

Reduz o número de padrões mas não perde informação de suporte!

O padrão fechado é uma compressão sem perdas de padrões frequentes. Você ainda será capaz de determinar: "{a2, ..., a40}: 2" e "{a5, a51}: 1" a partir dos padrões fechados, mantendo toda a informação de frequência necessária para análises subsequentes.

Expressando Padrões de Forma Comprimida: Max-Padrões

Solução 2: Max-Padrões

Um padrão X é um max-padrão se X é frequente e não existe nenhum super-padrão frequente $Y \supseteq X$

Exemplo TDB1

T1: {a1, ..., a50}; T2: {a1, ..., a100}

Com suporte mínimo = 1, quantos max-padrões TDB1 contém?

Apenas Um: P: "{a1, ..., a100}: 1"

Diferença dos Padrões Fechados

Não se preocupa com o suporte real dos sub-padrões de um max-padrão

Compressão com Perdas

- Sabemos que {a1, ..., a40} é frequente
- Mas não sabemos o suporte real de {a1, ..., a40}
- Perdemos informação detalhada de frequência

❏ **Conclusão Importante:** Em muitas aplicações, minerar padrões fechados é mais desejável do que minerar max-padrões, pois preserva informações críticas de suporte necessárias para tomada de decisão e análise detalhada.

Métodos Escaláveis de Mineração de Conjuntos de Itens Frequentes

O Princípio Apriori

Se {cerveja, fralda, nozes} é frequente, então {cerveja, fralda} também é frequente

Ou seja, toda transação contendo {cerveja, fralda, nozes} também contém {cerveja, fralda}

O princípio Apriori baseia-se na propriedade de fechamento descendente (downward closure) de conjuntos de itens frequentes: qualquer subconjunto de um conjunto frequente deve ser frequente.

Apriori

Abordagem de Geração e Teste de Candidatos

FP-Growth

Abordagem de Crescimento de Padrões Frequentes

ECLAT

Mineração com Formato de Dados Vertical

Geração de Candidatos

Algoritmo Apriori gera candidatos sistematicamente

Crescimento de Padrões

FP-Growth cresce padrões sem geração de candidatos

Formato Vertical

ECLAT usa representação eficiente dos dados

  **Leitura complementar:** Han, J., Cheng, H., Xin, D., Yan, X. (2007). "Frequent pattern mining: Current status and future directions", *Data Mining and Knowledge Discovery*, v. 15, n. 1, p. 55–86.

O Algoritmo Apriori - Um Exemplo

Banco de Dados TDB

Tid	Itens
10	A, C, D
20	B, C, E
30	A, B, C, E
40	B, E

Suporte mínimo (min_sup) = 2

1ª Varredura: C1 → L1

C1 (todos os candidatos): {A}:2, {B}:3, {C}:3, {D}:1, {E}:3

L1 (frequentes, sup ≥ 2): {A}:2, {B}:3, {C}:3, {E}:3

→ ~~{D}:1~~ eliminado por suporte insuficiente

2ª Varredura: C2 → L2

C2 (candidatos por junção de L1): {A,B}:1, {A,C}:2, {A,E}:1, {B,C}:2, {B,E}:3, {C,E}:2

L2 (frequentes, sup ≥ 2): {A,C}:2, {B,C}:2, {B,E}:3, {C,E}:2

→ ~~{A,B}:1~~ e ~~{A,E}:1~~ eliminados

3ª Varredura: C3 → L3

C3 (candidatos por junção de L2): {B,C,E}:2

→ ~~{A,B,C}~~ e ~~{A,C,E}~~ podados (subconjuntos infrequentes)

L3 (frequentes): {B,C,E}:2

O processo demonstra como o Apriori poda eficientemente o espaço de busca: D é eliminado na primeira varredura por não atingir o suporte mínimo, e apenas um 3-itemset é gerado na etapa final. Cada varredura refina progressivamente os candidatos até encontrar todos os padrões frequentes.

O Algoritmo Apriori (Pseudocódigo)

Algoritmo Apriori(T, ϵ)

Entradas:

T : banco de dados de transações

ϵ : limiar de suporte mínimo

Saída:

L : conjunto de todos os itemsets frequentes

$L_1 := \{\text{itemsets frequentes de tamanho } 1\};$

$k := 2;$

enquanto $L_{\{k-1\}} \neq \emptyset$ faça

$C_k := \text{gerar_candidatos}(L_{\{k-1\}});$

para cada transação $t \in T$ faça

$C_t := \text{subconjunto}(C_k, t);$

para cada candidato $c \in C_t$ faça

$c.\text{count} := c.\text{count} + 1;$

fim para

fim para

$L_k := \{c \in C_k \mid c.\text{count} \geq \epsilon\};$

$k := k + 1;$

fim enquanto

retornar $L := \bigcup_k L_k;$

Princípio de Poda Apriori

"Se existe algum conjunto de itens que é infrequente, seu superconjunto não deve ser gerado ou testado"

Scan Inicial do BD

Varredura única para obter conjunto de 1-itens frequentes

Geração de Candidatos

Gerar conjuntos candidatos de tamanho $k+1$ a partir dos frequentes de tamanho k

Teste Contra BD

Testar os candidatos comparando com o banco de dados

Término

Encerrar quando nenhum conjunto frequente ou candidato puder ser gerado

Este processo iterativo garante que apenas padrões promissores sejam considerados, reduzindo drasticamente o espaço de busca através da propriedade de anti-monotonicidade do suporte.

Implementação do Apriori

Estruturas de Dados Eficientes

Uso de tabelas hash para contagem rápida de suporte e estruturas de árvore para geração eficiente de candidatos

Processamento Paralelo

Particionamento do banco de dados para processamento distribuído e contagem paralela de candidatos

Otimizações de Memória

Técnicas de compressão e estruturas compactas para minimizar uso de memória durante a mineração

A implementação prática do Apriori requer cuidadosa consideração de trade-offs entre memória, velocidade e escalabilidade. Técnicas avançadas incluem poda de candidatos usando hash, ordenação de itens por frequência, e uso de bitmaps para representação compacta de transações.

Limitações Básicas do Método Apriori

Múltiplas Varreduras

Várias varreduras do banco de dados transacional são necessárias

Número Imenso de Candidatos

Geração explosiva de conjuntos candidatos em dados densos

Alto Custo de Contagem

Grande carga de trabalho para contar suporte dos candidatos

Gargalos da Abordagem Apriori

- **Busca em largura:** Abordagem nível a nível (level-wise)
- **Geração e teste:** Frequentemente gera número enorme de candidatos

Melhorias Propostas

- Reduzir passagens de varredura do banco
- Diminuir número de candidatos
- Facilitar contagem de suporte

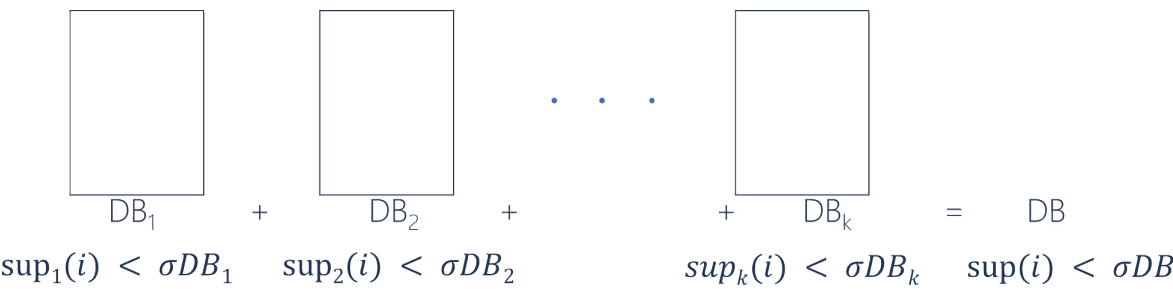
- ❑ Essas limitações motivaram o desenvolvimento de algoritmos alternativos como FP-Growth, que evita a geração de candidatos, e ECLAT, que usa formato vertical de dados para contagem mais eficiente.

Partition: Varredura do Banco de Dados Apenas Duas Vezes

Princípio Fundamental
Qualquer conjunto de itens que é potencialmente frequente no BD deve ser frequente em pelo menos uma das partições do BD

Varredura 1
Particionar banco de dados e encontrar padrões frequentes locais (preferencialmente cabendo na memória principal)

Varredura 2
Consolidar padrões frequentes globais verificando candidatos em todo o banco



A abordagem de particionamento reduz drasticamente o número de varreduras necessárias, tornando o algoritmo mais escalável para grandes volumes de dados. Cada partição pode ser processada independentemente, possibilitando paralelização eficiente e melhor utilização de recursos computacionais.

Amostragem para Padrões Frequentes

Técnica complementar de otimização do Apriori baseada em selecionar uma amostra representativa do banco de dados para minerar padrões com menor custo computacional.

1. Selecionar Amostra
Escolher amostra representativa **S** do BD com suporte mínimo reduzido para compensar o tamanho menor

2. Minerar Padrões
Buscar padrões frequentes em **S**, operando inteiramente na memória principal com alto desempenho

3. Verificar Resultados
Validar padrões encontrados contra o **BD completo** para confirmar suporte real e detectar falsos negativos

☐ **Vantagens:** redução de tempo e memória; **Desafios:** pode perder padrões raros e requer ajuste cuidadoso do suporte mínimo.

Pattern-Growth e Mineração Vertical

Uma introdução aos métodos avançados de mineração de padrões frequentes que superam as limitações das abordagens tradicionais baseadas em geração de candidatos.

Construção da FP-tree a Partir de um Banco de Dados Transacional

Primeira Varredura do BD

Escanear o banco de dados uma vez para encontrar todos os 1-itemsets frequentes (padrões de item único) que atendem ao suporte mínimo.

Ordenação por Frequência

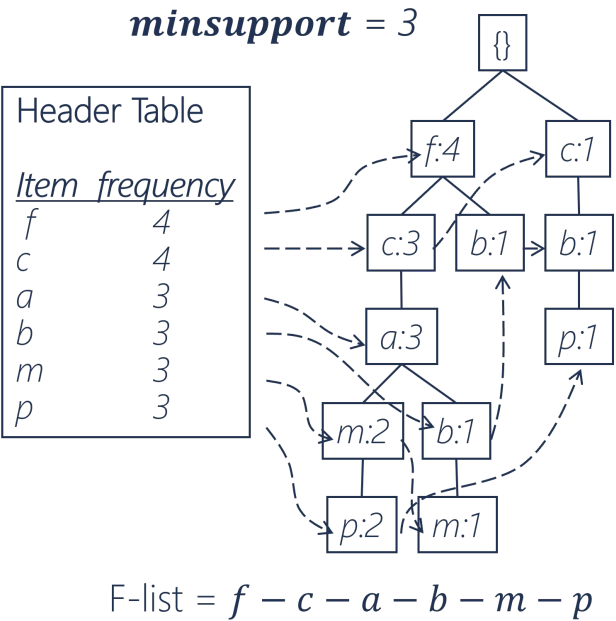
Ordenar os itens frequentes em ordem decrescente de frequência, criando a f-list (lista de frequência) que guiará a construção da árvore.

Segunda Varredura e Construção

Escanear o BD novamente, construindo a FP-tree inserindo cada transação ordenada conforme a f-list na estrutura de árvore.

Exemplo de Transformação de Transações

TID	Itens Comprados	Itens Frequentes Ordenados
100	f, a, c, d, g, i, m, p	f, c, a, m, p
200	a, b, c, f, l, m, o	f, c, a, b, m
300	b, f, h, j, o, w	f, b
400	b, c, k, s, p	c, b, p
500	a, f, c, e, l, p, m, n	f, c, a, m, p



Note como itens infrequentes (d, g, i, l, o, h, j, w, k, s, e, n) são eliminados e os itens restantes são reordenados pela frequência decrescente.

Particionamento de Padrões e Bancos de Dados

O FP-Growth particiona tanto o espaço de padrões quanto o banco de dados de forma recursiva. Esta estratégia de dividir e conquistar permite que o algoritmo processe subproblemas menores e mais gerenciáveis de forma independente.

Cada nó na FP-tree representa um prefixo de padrão, e a subárvore abaixo dele contém todas as extensões possíveis daquele prefixo. O algoritmo processa sistematicamente cada item frequente, criando bases de padrões condicionais que capturam apenas as transações relevantes para aquele item específico.

Vantagens do Particionamento

- Redução do espaço de busca
- Processamento paralelo potencial
- Menor uso de memória
- Eliminação de dados irrelevantes

Encontrando Padrões Contendo P a Partir do BD Condicional de P



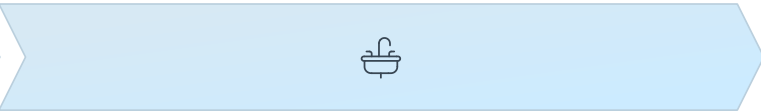
Iniciar na Tabela de Cabeçalhos

Começar pela tabela de cabeçalhos de itens frequentes na FP-tree



Percorrer a FP-tree

Seguir o link de cada item frequente p através da árvore



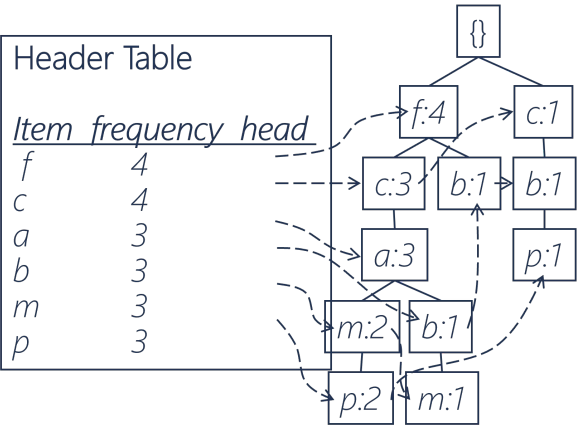
Somar Caminhos Prefixos

Agregar todos os caminhos prefixos transformados para formar a base de padrões condicionais

Bases de Padrões Condicionais

Para cada item frequente, extraímos sua base de padrões condicionais, que contém todos os prefixos de caminhos que levam a esse item, junto com suas contagens de suporte.

Item	Base de Padrão Condicional
p	fcam:2, cb:1
m	fca:2, fcab:1
b	fca:1, f:1, c:1
a	fc:3
c	f:3



Das Bases de Padrões Condicionais para FP-trees Condicionais

Processo de Construção

Para cada base de padrões, seguimos um processo sistemático:

1. **Acumular contagens:** Somar a contagem de cada item na base de padrões
2. **Filtrar itens frequentes:** Manter apenas itens que atendem ao suporte mínimo
3. **Construir FP-tree condicional:** Criar uma nova FP-tree apenas com os itens frequentes da base

Exemplo: Base de Padrão Condicional de m

m-conditional pattern base: fca:2, fcab:1

Após acumular as contagens: f:3, c:3, a:3, b:1

Se o suporte mínimo for 2, eliminamos b:1 e construímos a FP-tree condicional com f, c, e a.

Todos os Padrões Frequentes Relacionados a m

Padrões de 1 item

m

Padrões de 2 itens

fm, cm, am

Padrões de 3 itens

fcm, fam, cam

Padrões de 4 itens

fcam

O Método de Mineração Frequent Pattern Growth

Filosofia e Introdução do FP-Growth

O FP-Growth **revolucionou a mineração de padrões frequentes** ao introduzir uma busca em profundidade que evita completamente a geração explícita de candidatos, reduzindo drasticamente o custo computacional.

- Busca em profundidade (depth-first search)
- Eliminação da geração explícita de candidatos
- Estrutura de dados compacta e eficiente

A filosofia central é crescer padrões longos a partir de padrões curtos, utilizando apenas itens frequentes locais. O método constrói padrões de forma incremental, particionando o espaço de busca e evitando a explosão combinatória do Apriori.

Construir Bases Condicionais

Para cada item frequente, construir sua base de padrão condicional e sua FP-tree condicional correspondente

Processo Recursivo

Repetir o processo em cada FP-tree condicional recém-criada, minerando padrões de forma recursiva

Condição de Parada

Até que a FP-tree resultante esteja vazia ou contenha apenas um caminho único

Geração Final

Um caminho único gera todas as combinações de seus sub-caminhos, cada uma sendo um padrão frequente

Pseudocódigo

Algoritmo **FP-Growth**(BD, min_sup)
ENTRADA: BD (banco de dados transacional), min_sup (suporte mínimo)
PRÉ-CONDIÇÃO: min_sup > 0

PASSO 1 — Construir FP-tree

Varredura 1: contar frequência de cada item
Remover itens com suporte < min_sup
Ordenar itens por frequência decrescente
Varredura 2: inserir transações na FP-tree

PASSO 2 — Minerar FP-tree (recursivo)

PARA CADA item a_i na tabela de cabeçalhos
(do menos ao mais frequente):
 gerar padrão P = prefixo $\cup$ {a_i}
 com suporte = **suporte**(a_i)
 construir **base_condicional**(P)
 $\leftarrow$ prefixos de caminhos até a_i
 construir FP-tree_condicional
 a partir de **base_condicional**(P)
 SE FP-tree_condicional $\neq \emptyset$ ENTÃO
 FP-Growth(FP-tree_condicional,
 min_sup, P) $\leftarrow$ chamada recursiva
 SE FP-tree_condicional tem caminho
 único L ENTÃO
 gerar todos os subconjuntos de L
 como padrões frequentes

SAÍDA: conjunto completo de padrões frequentes com suporte

FP-Growth vs. Apriori: Escalabilidade com o Limiar de Suporte

A comparação de desempenho entre FP-Growth e Apriori revela diferenças significativas em escalabilidade, especialmente quando variamos o limiar de suporte mínimo.

Comportamento com Diferentes Limiares

À medida que o **limiar de suporte diminui**, o número de padrões frequentes aumenta exponencialmente. O Apriori sofre severamente com este crescimento porque:

- Gera um número massivo de candidatos
- Requer múltiplas varreduras do banco de dados
- Testa cada candidato individualmente contra o BD

Em contraste, o FP-Growth mantém desempenho superior porque trabalha com estruturas compactas e evita completamente a geração de candidatos, minerando padrões diretamente da FP-tree.

Vantagem do FP-Growth

10x

Mais Rápido

Em cenários de baixo suporte

2

Varreduras

Apenas duas leituras do BD

Os gráficos de desempenho demonstram consistentemente que o FP-Growth supera o Apriori, especialmente em bases de dados grandes e com limiares de suporte baixos, onde a diferença pode ser de ordem de magnitude.

 **Referência:** J. Han, J. Pei, and H. Tong, Data Mining: Concepts and Techniques, 4th edition. Cambridge, MA: Morgan Kaufmann, 2022.

Vantagens da Abordagem Pattern Growth

Dividir e Conquistar

Decompõe tanto a tarefa de mineração quanto o banco de dados de acordo com os padrões frequentes obtidos até o momento. Isso leva a uma **busca focada em bancos de dados menores**, cada um contendo apenas as transações relevantes para um contexto específico.

Banco de Dados Comprimido

A estrutura FP-tree serve como uma **representação compacta** do banco de dados original, eliminando informações redundantes e infrequentes enquanto preserva todas as informações necessárias para mineração completa de padrões.

Completude

A FP-tree preserva informação completa para mineração de padrões frequentes. Nunca quebra um padrão longo de qualquer transação, mantendo todas as relações entre itens intactas.

Eliminação de Candidatos

Não há geração de candidatos nem teste de candidatos. O algoritmo **minera padrões diretamente** da estrutura FP-tree, evitando completamente o overhead de criar e verificar combinações que podem não ser frequentes.

Varreduras Limitadas

Não há varredura repetida de todo o banco de dados. Apenas **duas varreduras completas** são necessárias: uma para determinar itens frequentes e outra para construir a FP-tree. Todo o processamento subsequente opera na estrutura em memória.

Compactação

Elimina itens infrequentes, ordena itens por frequência decrescente para maximizar compartilhamento de prefixos, e transações com prefixos comuns compartilham nós, reduzindo redundância.

ECLAT: Mineração Explorando Formato Vertical de Dados

O algoritmo ECLAT (Equivalence Class Clustering and bottom-up Lattice Traversal) representa uma abordagem alternativa para mineração de padrões frequentes, baseada em uma **representação vertical dos dados** ao invés da representação horizontal tradicional.

Formato Vertical de Dados

No formato vertical, cada item é associado a uma lista de identificadores de transações (TID-list) onde ele aparece. Esta transformação permite operações eficientes de interseção para determinar suporte de itemsets.

Exemplo de Representação Vertical:

- Item A: {TID1, TID2, TID5}
- Item B: {TID2, TID3, TID4}
- Item C: {TID1, TID2, TID3}

Para encontrar o suporte de {A,B}, basta fazer a interseção: {TID2}

O ECLAT é particularmente eficiente quando o banco de dados pode ser mantido em memória e quando há muitas transações curtas com itens que se repetem frequentemente.

Algoritmo ECLAT(BD, min_sup)

ENTRADA: BD (banco de dados transacional), min_sup (suporte mínimo)

PRÉ-CONDIÇÃO: min_sup > 0

PASSO 1 — Conversão para Formato Vertical

PARA CADA transação $t \in \text{BD}$:

PARA CADA item $i \in t$:

$\text{TID-list}(i) \leftarrow \text{TID-list}(i) \cup \{t.\text{id}\}$

Remover itens com $|\text{TID-list}(i)| < \text{min_sup}$

PASSO 2 — Mineração por Equivalência de Classes

Inicializar classes de equivalência por prefixo comum

PARA CADA classe de equivalência E com prefixo P:

PARA CADA par $(P \cup \{a\}, P \cup \{b\})$ em E:

$\text{TID-list}(P \cup \{a,b\}) \leftarrow \text{TID-list}(P \cup \{a\}) \cap \text{TID-list}(P \cup \{b\})$

SE $|\text{TID-list}(P \cup \{a,b\})| \geq \text{min_sup}$ ENTÃO

adicionar $P \cup \{a,b\}$ aos padrões frequentes

criar nova subclasse para extensões futuras

SAÍDA: conjunto de itemsets frequentes com TID-lists

❑ **Vantagens do ECLAT:** Interseção de conjuntos computacionalmente eficiente · Ideal para datasets densos com muitos itens frequentes · Busca em profundidade evita geração de candidatos desnecessários · Subproblemas independentes permitem fácil paralelização

❑ **Referência:** M.J. Zaki, 2000, Scalable algorithms for association mining, *IEEE Transactions on Knowledge and Data Engineering*, v. 12, n. 3, p. 372–390.

Quando Usar FP-Growth ou ECLAT?

A escolha entre FP-Growth e ECLAT depende fundamentalmente das características específicas dos seus dados.

FP-Growth: Bancos de Dados Grandes e Esparsos

O FP-Growth é a escolha ideal quando você tem:

- **Grandes volumes de transações** com muitos itens diferentes
- **Dados esparsos** onde cada transação contém apenas uma pequena fração dos itens possíveis
- **Memória limitada** para armazenar TID-lists (o ECLAT pode consumir muita memória em dados esparsos)
- **Necessidade de compactação** - a FP-tree comprime eficientemente dados redundantes

Exemplos: transações de varejo com milhares de produtos, logs de navegação web, registros de compras online.

Fatores de Decisão

1. **Densidade dos dados:** esparso → FP-Growth; denso → ECLAT
2. **Tamanho das transações:** longas → FP-Growth; curtas → ECLAT
3. **Memória disponível:** limitada → FP-Growth; abundante → ECLAT
4. **Infraestrutura:** single-core → FP-Growth; multi-core → ECLAT

ECLAT: Datasets Densos com Interseções Eficientes

O ECLAT supera quando você tem:

- **Dados densos** onde muitos itens aparecem em muitas transações
- **Transações curtas** com poucos itens cada
- **Memória suficiente** para manter TID-lists em RAM
- **Possibilidade de paralelização** - ECLAT é naturalmente paralelizável

Exemplos: análise de cesta de mercado em lojas especializadas, análise de sintomas médicos, dados de sensores com poucos estados possíveis.

Na Prática

Experimente ambos os algoritmos com uma amostra dos seus dados e compare:

- Tempo de execução
- Uso de memória
- Escalabilidade

Regras de Associação e Tópicos Avançados

Explorando técnicas avançadas para descoberta de padrões significativos em grandes conjuntos de dados e medidas de interesse para avaliação de regras de associação.

Análise de Regras

A análise de regras de associação é um componente fundamental da mineração de dados, permitindo descobrir relações interessantes entre variáveis em grandes bases de dados transacionais. Este processo envolve a identificação de padrões frequentes e a avaliação de sua significância através de diversas métricas.

O desafio central está em distinguir regras verdadeiramente úteis de associações espúrias ou triviais, demandando medidas robustas de interesse que capturem tanto a força estatística quanto a relevância prática das descobertas.

	Café	¬ Café	Soma(linha)
Leite	2000	1750	3750
¬ Leite	1000	250	1250
Soma(col.)	3000	2000	5000

Regras fortes nem sempre são interessantes

Observe o conjunto de dados:

beber café → beber leite [40%, 66,7%]

é enganoso. A porcentagem geral de clientes que bebem leite é **75% > 66,7%**.

beber café → ¬ beber leite [20%, 33,3%]

é mais precisa, embora com menor suporte e confiança

Medidas de Interesse para Regras de Associação

As medidas de interesse são fundamentais para avaliar a qualidade e relevância das regras de associação descobertas. Diferentes métricas capturam aspectos distintos das relações entre itens, desde correlação estatística até utilidade prática.

Suporte

Frequência com que os itens aparecem juntos no conjunto de dados. Indica a popularidade da regra.

Confiança

Probabilidade condicional de Y dado X. Mede a força preditiva da regra $X \rightarrow Y$.

Lift

Razão entre a confiança observada e a esperada sob independência. Valores > 1 indicam correlação positiva.

$$lift = \frac{P(X \rightarrow Y)}{P(X)P(Y)}$$

	Café	¬ Café	Soma(linha)
Leite	2000	1750	3750
¬ Leite	1000	250	1250
Soma(col.)	3000	2000	5000

Café → Leite

$$lift(Café, Milk) = \frac{(2000/5000)}{(3000/5000 \bullet 3750/5000)} = 0,89$$

Café → ¬Leite

$$lift(Café, \neg Leite) = \frac{(1000/5000)}{(3000/5000 \bullet 1250/5000)} = 1,33$$

Leituras complementares: Geng & Hamilton (2006), "Interestingness measures for data mining: A survey", *ACM Computing Surveys*, v. 38, n. 3 · McGarry (2005), "A survey of interestingness measures for knowledge discovery", *Knowledge Engineering Review*, v. 20, n. 1, p. 39–61.

Tabela de Contingência com Valores Esperados

A tabela de contingência é uma ferramenta essencial para calcular medidas de interesse, apresentando a distribuição observada de transações contendo diferentes combinações de itens. Os valores esperados são calculados assumindo independência estatística entre os itens.

Esta abordagem permite comparar frequências observadas com frequências esperadas, revelando associações que são estatisticamente significativas. A diferença entre valores observados e esperados é a base para muitas medidas de interesse, incluindo qui-quadrado e lift.

$$\chi^2 = \sum \frac{(Observado - Esperado)^2}{Esperado} = \frac{(f_{11} - E_{11})^2}{E_{11}} + \frac{(f_{10} - E_{10})^2}{E_{10}} + \frac{(f_{01} - E_{01})^2}{E_{01}} + \frac{(f_{00} - E_{00})^2}{E_{00}}$$

Valores Observados:

	Café(c)	Café(¬c)	Total
Leite (m)	150	50	200
¬Leite (¬m)	650	150	800
Total	800	200	1000

$$\chi^2 = \frac{(150 - 160)^2}{160} + \frac{(50 - 40)^2}{40} + \frac{(650 - 640)^2}{640} + \frac{(150 - 160)^2}{160}$$
$$\chi^2 = 0.625 + 2.5 + 0.156 + 0.625 = 3.906$$

Conclusão: Com $\chi^2 = 3.906$ e 1 grau de liberdade, o valor é relativamente baixo, sugerindo uma associação fraca entre leite e café neste exemplo.

Problema das Transações Nulas

Quando muitas transações não contêm nenhum dos itens da regra, certas medidas de interesse podem ser infladas artificialmente, comprometendo a análise.

Impacto no Cálculo

Medidas baseadas em correlação, como o χ^2 e o lift, incluem no denominador a frequência de transações nulas (f_{00}). Quando f_{00} é muito grande, esses valores podem crescer artificialmente, sugerindo uma associação forte mesmo quando os itens raramente co-ocorrem.

Exemplo Prático

Em um supermercado com 10.000 produtos, se apenas 100 transações contêm leite ou café, as outras 9.900 são transações nulas para essa regra. Essa massa de f_{00} distorce métricas de correlação, exigindo medidas invariantes a transações nulas, como Kulczynski e IR.

- ❏ Medidas invariantes a transações nulas ignoram f_{00} no cálculo, focando apenas nas co-ocorrências efetivas entre os itens.

Medidas Resistentes ao Problema de Transações Nulas

Algumas medidas de interesse são mais robustas ao problema das transações nulas, focando apenas nas transações que contêm pelo menos um dos itens relevantes. Estas métricas proporcionam avaliações mais precisas da força de associação em contextos de alta dimensionalidade.

Medidas como Kulczynski e IR (Imbalance Ratio) são exemplos de métricas que minimizam o impacto das transações nulas, concentrando-se nas co-ocorrências efetivas entre itens ao invés de considerar todas as ausências conjuntas.

Kulczynski (Kulc)

$$Kulc(A, B) = \frac{1}{2} \times \left(\frac{f_{11}}{f_{10} + f_{11}} + \frac{f_{11}}{f_{01} + f_{11}} \right)$$

Média das probabilidades condicionais $P(B|A)$ e $P(A|B)$. Medida simétrica e resistente a transações nulas. Varia de 0 a 1.

IR (Imbalance Ratio)

$$IR(A, B) = \frac{|f_{10} - f_{01}|}{f_{10} + f_{01} + f_{11}}$$

Mede o desequilíbrio entre as ocorrências de A e B. IR = 0 indica equilíbrio perfeito; IR = 1 indica desequilíbrio total. Usado em conjunto com Kulc para caracterizar a natureza da associação.

Onde:

f_{11} = transações contendo A e B

f_{01} = transações contendo B mas não A

f_{10} = transações contendo A mas não B

f_{00} = transações sem A nem B (ignorado nestas medidas)

Comparação de Medidas de Interesse: Leite (m) e Café (c)

A escolha da medida de interesse apropriada depende do objetivo da análise e das características dos dados. Diferentes métricas podem produzir rankings distintos das mesmas regras, destacando aspectos diferentes das associações.

Conjunt o	f ₁₁	f ₁₀	f ₀₁	Kulc	IR	Interpretação
D1	10000	1000	1000	0,91	0,00	Positivamente correlacionados, equilibrados
D2	10000	1000	100	0,91	0,09	Positivamente correlacionados, levemente desequilibrados
D3	10000	1000	10	0,91	0,47	Positivamente correlacionados, desequilibrados
D4	10000	1000	1	0,91	0,83	Positivamente correlacionados, muito desequilibrados

O critério combinado **Kulc + IR** proposto por Han oferece uma caracterização completa da associação entre dois itens:

Kulc mede a força da associação (próximo de 1 = forte, próximo de 0 = fraca), enquanto IR mede o desequilíbrio entre os itens (próximo de 0 = equilibrado, próximo de 1 = desequilibrado). Juntos, fornecem uma caracterização completa da associação.

Nos exemplos D1 a D4, o Kulc permanece constante em **0,91** — indicando associação forte em todos os casos — enquanto o IR aumenta progressivamente, revelando crescente desequilíbrio nas ocorrências individuais de leite (m) e café (c).

A análise comparativa revela que nenhuma medida única é universalmente superior. A seleção adequada requer compreensão dos vieses de cada métrica e do contexto específico da aplicação.

Padrões Top-k com Consciência de Redundância

Por que padrões top-k conscientes de redundância?

A mineração tradicional de padrões frequentes pode gerar conjuntos massivos de regras, muitas das quais são redundantes ou fornecem informações similares. O desafio é identificar um subconjunto compacto e diversificado de padrões que maximize a informação útil.

Alta Significância

Padrões devem ser estatisticamente fortes e interessantes.

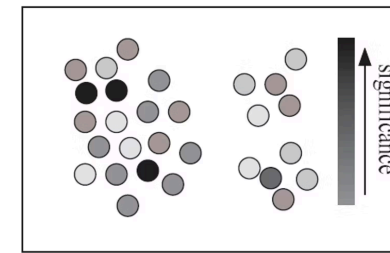
Baixa Redundância

Minimizar sobreposição de informação entre padrões selecionados.

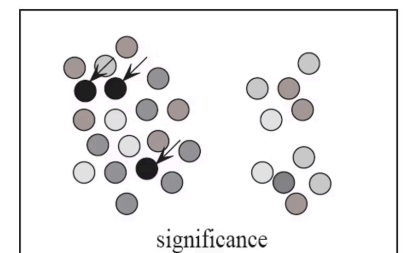
MMS: Significância Marginal Máxima

Métrica combinada que equilibra relevância e diversidade.

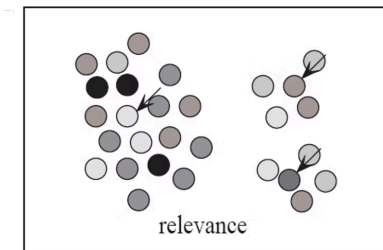
Extraindo Padrões



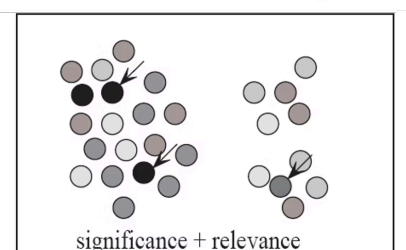
a set of patterns



traditional top- k



summarization



redundancy-aware

Mineração de Regras de Associação Multi-Nível

Em muitas aplicações práticas, os itens não existem isoladamente, mas formam hierarquias naturais. Por exemplo, em supermercados, produtos específicos pertencem a categorias que, por sua vez, pertencem a departamentos maiores.

Hierarquias Naturais

Itens organizados em taxonomias com múltiplos níveis de abstração, do geral ao específico.

Suporte Flexível

Configurações adaptativas de suporte considerando que itens específicos naturalmente têm menor frequência.

Mineração Compartilhada

Exploração de padrões em diferentes níveis de granularidade simultaneamente.

Suporte Reduzido

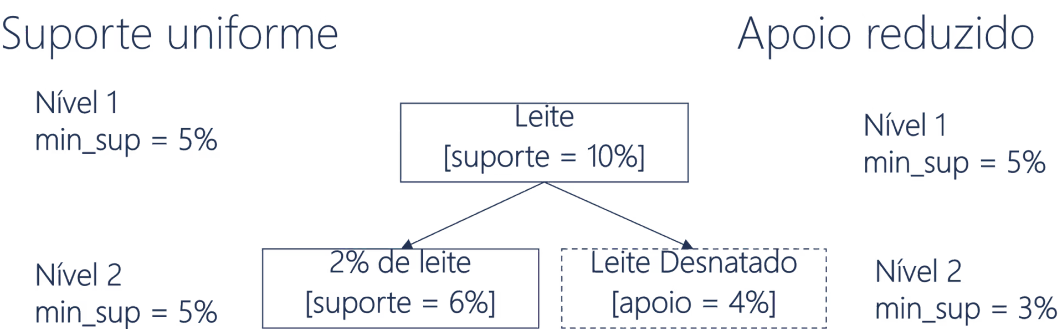
Itens em níveis inferiores da hierarquia requerem limiares de suporte proporcionalmente menores para serem considerados frequentes.

Associação Multi-Nível: Suporte Flexível e Filtragem de Redundância

Estratégias de Suporte

A definição de limiares de suporte apropriados para cada nível da hierarquia é crucial. Um limiar uniforme resultaria em perda de padrões específicos interessantes nos níveis inferiores, ou em explosão de padrões triviais nos níveis superiores.

A abordagem de **suporte reduzido** aplica limiares decrescentes conforme descemos na hierarquia, permitindo descobrir associações específicas sem sobrecarregar o algoritmo.



Filtragem de Redundância

Regras em níveis diferentes podem representar essencialmente a mesma informação. Por exemplo, "pão → manteiga" e "pão integral → manteiga com sal" podem ser redundantes se a segunda não adiciona informação significativa.

Ancestralidade

Eliminar regras se versões mais gerais já foram encontradas.

Especificidade

Priorizar regras mais específicas quando fornecem insights adicionais.

Mineração de Associações Multi-Dimensionais

Além de hierarquias de produtos, dados transacionais frequentemente contêm múltiplas dimensões que podem ser analisadas simultaneamente. Cada transação pode ser caracterizada por atributos como localização, tempo, características demográficas do cliente, e método de pagamento.

Dimensões Múltiplas

Análise considerando simultaneamente produto, localização, tempo, perfil do cliente, e outras dimensões contextuais para descobrir padrões mais ricos.

Regras Inter-Dimensionais

Descoberta de associações que cruzam dimensões diferentes, como "clientes jovens em áreas urbanas compram smartphones com fones premium".

Segmentação Contextual

Identificação de padrões válidos apenas em contextos específicos, permitindo personalização e estratégias direcionadas.

Esta abordagem expande significativamente o espaço de busca, mas também permite descobrir insights mais acionáveis e contextualizados para tomada de decisão.

Padrões Negativos e Raros

Padrões Raros

Alguns padrões têm suporte extremamente baixo mas são altamente relevantes para o negócio. A compra de itens de luxo como relógios Rolex, por exemplo, é rara mas representa valor significativo.

- **Definição**

Suporte muito baixo mas interessante para análise e estratégia.

- **Abordagem**

Configurar limiares de suporte individuais ou baseados em grupos especiais para itens valiosos.

Insight Chave: Padrões negativamente correlacionados que são *infrequentes* tendem a ser mais interessantes do que aqueles que são frequentes, revelando substituições e incompatibilidades importantes.

Padrões Negativos

Associações negativas revelam itens que raramente são comprados juntos, indicando correlação negativa. Por exemplo, é improvável que alguém compre simultaneamente um Ford Expedition (SUV) e um Toyota Prius (híbrido).

De Padrões ao Conhecimento Acionável

O objetivo final da mineração de regras de associação não é simplesmente gerar listas extensas de padrões, mas transformar dados brutos em insights acionáveis que impulsionem decisões estratégicas e operacionais.

Geração Massiva

Muitas regras podem ser geradas automaticamente a partir dos dados transacionais.

Filtragem Inteligente

Medidas de interesse ajudam a filtrar e priorizar regras potencialmente úteis.

Interpretação Especializada

Conhecimento de domínio é essencial para interpretar e validar descobertas.

Ação Estratégica

Transformar insights em estratégias concretas de negócio e operações.

📌 **Lembre-se:** A mineração de dados é um processo colaborativo entre algoritmos estatísticos e expertise humana. As medidas quantitativas fornecem direção, mas o valor real emerge quando especialistas de domínio interpretam os padrões no contexto específico do negócio.

📌 **Leitura complementar:** Luna, J. M., Fournier-Viger, P., Ventura, S. (2019). "Frequent itemset mining: A 25 years review", *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, v. 9, n. 6.

Mineração de Padrões Sequenciais

Uma jornada através de algoritmos e técnicas para descoberta de padrões em dados sequenciais

Descobrimdo Padrões Frequentes em Sequências

Sequential Pattern Mining é uma técnica fundamental de mineração de dados que busca **identificar o conjunto completo de subsequências frequentes** dentro de um banco de dados de sequências. Dado um limiar mínimo de suporte (frequência), o objetivo é encontrar todos os padrões que satisfazem esse critério.

Uma **sequência** é uma lista ordenada de conjuntos de itens. Por exemplo: **<(ef)(ab)(df)cb>** representa uma sequência onde cada elemento pode conter múltiplos itens. Os itens dentro de um elemento são não-ordenados e listados alfabeticamente.

Considere que **<a(bc)dc>** é uma *subsequência* de **<a(abc)(ac)d(cf)>**. Com um limiar de suporte mínimo de 2, **<(ab)c>** seria considerado um *padrão sequencial* se aparecer em pelo menos 2 sequências do banco de dados.

Banco de Dados de Sequências

SID	Sequência
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

Sequential Pattern Mining: Principais Abordagens

Conceito Inicial

Introdução do conceito e algoritmo inicial baseado em Apriori por Agrawal & Srikant para mineração de padrões sequenciais

Requisitos Essenciais

Eficiência, escalabilidade, completude, mínimas varreduras no banco de dados e capacidade de incorporar restrições específicas do usuário

Algoritmos Representativos

- GSP (Generalized Sequential Patterns)
- SPADE (formato vertical)
- PrefixSpan (crescimento de padrões)

Abordagens Avançadas

Mineração de padrões sequenciais com restrições permite a incorporação de conhecimento específico do domínio para otimizar a busca

Padrões Fechados

CloSpan foca na mineração de padrões sequenciais fechados, reduzindo redundância e melhorando eficiência

A Propriedade Apriori em Padrões Sequenciais

A **propriedade Apriori** é o princípio fundamental que possibilita a poda eficiente de candidatos durante a mineração de padrões sequenciais. Esta propriedade estabelece que: se uma sequência não é frequente, então nenhuma de suas super-sequências pode ser frequente.

Conjunto de Dados

Analisa todas as sequências no banco de dados com suporte mínimo definido

Filtragem Apriori

Elimina candidatos cujas subsequências não atendem o limiar mínimo

Padrões Válidos

Retém apenas padrões que satisfazem $\text{min_sup} = 2$ ou superior

Exemplo: Com **suporte mínimo = 2**, se uma sequência <a> não aparece em pelo menos 2 transações, qualquer sequência mais longa contendo <a> como parte também não será frequente e pode ser descartada.

Generalized Sequential Pattern Mining

O **algoritmo GSP** (Generalized Sequential Pattern) representa uma evolução importante na mineração de padrões sequenciais, utilizando a propriedade Apriori para poda eficiente de candidatos.

Metodologia do GSP

Inicialização

Cada item individual no banco de dados é considerado um candidato de comprimento 1

Iteração por Nível

Para cada nível k , varredura do banco de dados para coletar contagem de suporte de cada sequência candidata

Geração de Candidatos

Gera sequências candidatas de comprimento $k+1$ a partir de sequências frequentes de comprimento k usando Apriori

Critério de Parada

Repete até que não haja mais sequências frequentes ou candidatos possam ser encontrados

Algoritmo GSP(BD, min_sup)

ENTRADA: BD (banco de dados de sequências),

min_sup (suporte mínimo)

PRÉ-CONDIÇÃO: min_sup > 0

PASSO 1 — Inicialização

Varredura 1: contar suporte de cada item

$L_1 \leftarrow \{\text{sequências de comprimento 1 com suporte} \geq \text{min_sup}\}$

$k \leftarrow 2$

PASSO 2 — Iteração por Nível

ENQUANTO $L_{\{k-1\}} \neq \emptyset$ FAÇA:

$C_k \leftarrow \text{gerar_candidatos}(L_{\{k-1\}})$

PARA CADA sequência $s \in \text{BD}$:

incrementar contagem de candidatos em C_k contidos em s

$L_k \leftarrow \{c \in C_k \mid \text{suporte}(c) \geq \text{min_sup}\}$

$k \leftarrow k + 1$

PASSO 3 — Geração de Candidatos

PARA CADA par $(s_1, s_2) \in L_{\{k-1\}}$

com mesmo prefixo de tamanho $k-2$:

$c \leftarrow s_1 \oplus \text{último_elemento}(s_2) \leftarrow \text{junção}$

SE todos sub-padrões de $c \in L_{\{k-1\}}$

adicionar c a C_k ← poda Apriori

SAÍDA: $L = \bigcup_k L_k$ (sequências frequentes)

📌 **Principal Vantagem:** A poda de candidatos através da propriedade Apriori reduz drasticamente o espaço de busca, tornando o algoritmo computacionalmente viável para grandes conjuntos de dados.

GSP: Geração de Candidatos

A **geração de candidatos** no algoritmo GSP é um processo sistemático que combina sequências frequentes de comprimento k para criar candidatos de comprimento $k+1$. Este processo utiliza duas estratégias principais: junção de itens isolados e fusão de itens dentro de elementos.

A eficiência do GSP está diretamente relacionada à sua capacidade de gerar apenas candidatos viáveis, eliminando combinações que certamente não serão frequentes através da aplicação rigorosa da propriedade Apriori.

Encontrando Padrões Sequenciais de Comprimento 1

Vamos examinar o **funcionamento do GSP** através de um exemplo detalhado. O processo começa com a identificação de padrões de comprimento 1, onde cada item individual é avaliado.

Candidatos Iniciais

Todas as sequências singleton são consideradas: **<a>**, ****, **<c>**, **<d>**, **<e>**, **<f>**, **<g>**, **<h>**

Realizamos uma **única varredura no banco de dados** para contar o suporte de cada candidato. Este passo é crucial pois estabelece a base para todas as iterações subsequentes.

<i>min_sup = 2</i>	
Seq. ID	Sequência
10	<(bd)cb(ac)>
20	<(bf)(ce)b(fg)>
30	<(ah)(bf)abf>
40	<(be)(ce)d>
50	<a(bd)bcb(ade)>

Contagem de Suporte

Candidato	Suporte
<a>	3
	5
<c>	4
<d>	3
<e>	3
<f>	2
<g>	1
<h>	1

Candidatos **<g>** e **<h>** são eliminados por não atingirem o suporte mínimo.

GSP: Gerando Candidatos de Comprimento 2

A geração de candidatos de comprimento 2 demonstra a **eficiência da propriedade Apriori**. Partindo dos 6 itens frequentes identificados, exploramos duas estratégias de combinação.

Itens Isolados

Combinações onde itens aparecem em elementos separados: **<ab>**, **<ac>**, **<ad>**, etc. Isso gera uma matriz 6×6 = **36 candidatos**

	<a>		<c>	<d>	<e>	<f>
<a>	<aa>	<ab>	<ac>	<ad>	<ae>	<af>
	<ba>	<bb>	<bc>	<bd>	<be>	<bf>
<c>	<ca>	<cb>	<cc>	<cd>	<ce>	<cf>
<d>	<da>	<db>	<dc>	<dd>	<de>	<df>
<e>	<ea>	<eb>	<ec>	<ed>	<ee>	<ef>
<f>	<fa>	<fb>	<fc>	<fd>	<fe>	<ff>

Itens Fundidos

Combinações onde itens aparecem no mesmo elemento: **<(ab)>**, **<(ac)>**, **<(ad)>**, etc. Isso gera 6×5/2 = **15 candidatos**

	<a>		<c>	<d>	<e>	<f>
<a>		<(ab)>	<(ac)>	<(ad)>	<(ae)>	<(af)>
			<(bc)>	<(bd)>	<(be)>	<(bf)>
<c>				<(cd)>	<(ce)>	<(cf)>
<d>					<(de)>	<(df)>
<e>						<(ef)>
<f>						

Impacto da Poda Apriori: Sem a propriedade Apriori, teríamos: $8 \times 8 + 8 \times 7 / 2 = 92$ candidatos. Com Apriori, geramos apenas: 51 candidatos

Esta redução significativa no espaço de busca é fundamental para a viabilidade computacional do algoritmo em conjuntos de dados reais de grande escala.

Graph Pattern Mining

A **mineração de padrões em grafos** estende os conceitos de padrões sequenciais para estruturas mais complexas. Um subgrafo é considerado frequente se sua frequência de ocorrência em um conjunto de dados atinge ou supera um limiar mínimo de suporte.



Estruturas Bioquímicas

Descoberta de padrões moleculares e compostos químicos em bases de dados farmacêuticas e biotecnológicas



Análise de Fluxo de Controle

Identificação de padrões em programas de computador para otimização e detecção de vulnerabilidades



Estruturas XML e Web

Mineração de comunidades e padrões estruturais em documentos XML e redes sociais online



Blocos de Construção

Fundamentos para classificação, clustering, compressão, comparação e análise de correlação em grafos

Subgrafos Frequentes na Prática

Considere um **conjunto de dados de grafos** onde buscamos identificar estruturas que aparecem com frequência mínima de 2 ocorrências. Este exemplo demonstra como padrões estruturais comuns podem ser extraídos de múltiplos grafos.

Conjunto de Dados de Grafos

Três grafos distintos **(A), (B) e (C)** compõem nosso conjunto de dados. Cada grafo representa uma estrutura de conectividade entre nós, podendo representar moléculas, redes sociais ou estruturas de dados.

Padrões Frequentes Descobertos

Com **suporte mínimo = 2**, identificamos padrões estruturais **(1) e (2)** que aparecem em pelo menos dois dos grafos originais.

Identificação de Motifs

Subgrafos frequentes representam motifs estruturais que se repetem, revelando padrões fundamentais na organização dos dados

Aplicação Prática

Estes padrões podem ser utilizados para classificação, predição de propriedades ou compreensão de princípios organizacionais subjacentes

Escalabilidade

Algoritmos eficientes são essenciais pois o espaço de subgrafos possíveis cresce exponencialmente com o tamanho dos grafos

Referências Principais

Esta seleção de referências representa os pilares fundamentais para o estudo aprofundado de mineração de dados, cobrindo desde conceitos básicos até técnicas avançadas e aplicações contemporâneas.



1. **J. Han, J. Pei, and H. Tong**, *Data Mining: Concepts and Techniques*, 4th edition. Cambridge, MA: Morgan Kaufmann, 2022.
2. **G. M. James, D. Witten, T. Hastie, and R. Tibshirani**, *An Introduction to Statistical Learning: With Applications in R*. Springer Nature, 2021.
3. **S. Garcia, J. Luengo, and F. Herrera**, *Data Preprocessing in Data Mining*. Springer, 2014.
4. **C. M. Bishop and H. Bishop**, *Deep Learning: Foundations and Concepts*. Springer Nature, 2023.